

Concept — The World Model

Date: 2026.09.04

| *A living model of our understanding of the world.*

Introduction

Modern product teams operate in an increasingly complex and rapidly changing environment. AI is accelerating what teams can build, leading to an increase in both product complexity and competition. The trends today mirror those of the 1960s: as machines began to become more powerful, the growing ambition of projects created an unmanageable system complexity at the human and software level:

To put it in another way: as the power of available machines grew by a factor of more than a thousand, society's ambition to apply these machines grew in proportion, and it was the poor programmer who found his job in this exploded field of tension between ends and means. The increased power of the hardware, together with the perhaps even more dramatic increase in its reliability, made solutions feasible that the programmer had not dared to dream about a few years before. And now, a few years later, he had to dream about them and, even worse, he had to transform such dreams into reality!

— Dijkstra

This general acceleration is also increasing the sheer volume of information from rapid experiments, product analytics, market developments, customer conversations, support tickets, etc., making it difficult to distinguish critical information from irrelevant noise. Teams must continually revisit their assumptions as the conditions behind earlier decisions change, but often they have a fragmented and outdated understanding of their operating environment, making it difficult to determine where to focus and how to act.

1. The new bottleneck

As AI makes code faster to generate, the development bottleneck increasingly moves upstream of implementation: deciding what to build, synthesizing requirements, exploring ideas through prototypes, and evaluating whether proposed changes address a real need. Teams can execute quickly while still working from incomplete evidence or assumptions; the prevalent failure mode today is building more than can be coherently organized or assimilated.

The software industry has developed a sophisticated lifecycle around source code management. Version control preserves changes, branches allow alternatives to be explored, reviews govern integration, and CI/CD standardizes the delivery. But the reasoning that gives rise to the code, however, is often much less connected. Customer feedback lives in support systems and call transcripts, requirements in documents and issue trackers, and important assumptions in conversations or people's heads.

These records form chains of reasoning: a customer report informs an explanation, that explanation motivates a requirement, and the requirement shapes an implementation. Documents may have version histories and decisions may be recorded, but the connections between them often depend on memory and manual documentation. As the chains grow, maintaining those connections becomes harder. When new evidence challenges an earlier explanation, a team must reconstruct which requirements and decisions relied on it. The rationale can disappear while the requirement remains. These conditions are elaborated in Product Development — The Information Environment.

Very often, there is no single source of product truth, resembling the period of the 2000s before the continuous integration became commonplace:

In order to integrate easily, any developer needs to be able to get a full set of current sources easily. There's nothing worse than having to go around to different developers to ask for the latest bits and then having to copy them over, figure out where to put them, all this before you can begin the build.

The standard is simple. Anyone should be able to bring a clean machine, connect it to the network, and with a single command bring down every source file that's needed to build the system under development.

— Fowler and Foemmel, *"The Benefits of Continuous Integration."*

We can hear in their description the familiar process of asking another engineer how or why a something was done a certain way; of being unable to determine a decision's rationale, and of having no recourse to correct it without a blind choice:

One of the hardest things to track during the life of a project is the motivation behind certain decisions. A new person coming on to a project may be perplexed, baffled, delighted, or infuriated by some past decision. Without understanding the rationale or consequences, this person has only two choices:

(1) Blindly accept the decision. This response may be OK, if the decision is still valid. It may not be good, however, if the context has changed and the decision should really be revisited. If the project accumulates too many decisions accepted without understanding, then the development team becomes afraid to change anything and the project collapses under its own weight.

(2) Blindly change it. Again, this may be OK if the decision needs to be reversed. On the other hand, changing the decision without understanding its motivation or consequences could mean damaging the project's overall value without realizing it. (E.g., the decision supported a non-functional requirement that hasn't been tested yet.)

— Nygard

<!--

The dependency-fragility framing draws on Andrew's reading of Exa's Canon article: <https://exa.ai/blog/composing-a-search-engine>

See the inspiration comment in `product-development-environment.md` and the selected excerpt in `../sources/exa-canon-excerpt.md`. Preserve this connection for further exploration of explicit dependencies and inspectable traces in reasoning systems.

-->

The opportunity is to give this upstream work a similarly explicit lifecycle: preserve the evidence behind a requirement, compare competing hypotheses, review proposed changes to the team's understanding, and trace their consequences through to implementation and observed results. Automated checks could identify missing sources or decisions that depend on a revised belief; judging whether the evidence warrants a conclusion would still require a separate evaluation.

2. The World Model

The world-model described by Jack Dorsey suggests how AI could maintain a continuously updated model of a company and its customers, drawing on decisions, discussions, code, plans, and observed customer behavior. These records become the basis for a shared understanding that guides action. This raises a practical question: how do we connect those observations to the hypotheses, beliefs, and decisions they inform, and keep that understanding current as new evidence arrives?

Our hypothesis is that a graph connecting observations, evidence, hypotheses, inferences, and beliefs could make that understanding explicit and maintainable. A team could inspect why it believes something, see which decisions depend on it, and identify what needs revisiting when the evidence changes. That is the capability we want to demonstrate: *making more of what a team learns usable in what it does next*.

Background agents could continuously extract relevant observations from conversations and events, guided by the team's goals, pending decisions, and current beliefs. A detail overlooked in a customer call might challenge an assumption behind a product decision. By connecting that observation to

the inference it bears on and retaining its source, the system could surface information that would otherwise go unused and keep the team's understanding current.